Module 2 – Introduction to Programming

- Overview of C Programming

THEORY EXERCISE: Write an essay covering the history and evolution of C programming. Explain its importance and why it is still used today.

C programming language was developed in the 1970s by Dennis Ritchie at Bell Labs. It was created as an improved version of the B language and mainly used to develop the UNIX operating system. Later, standard versions like ANSI C (1989), C99, and C11 were introduced.

Importance of C

C is a powerful and basic programming language. It is fast, efficient, and works close to hardware. Many modern languages like C++ and Java are influenced by C, so it is called the "mother language."

Why C is Still Used Today

C is still used because it is fast, portable, and efficient. It is widely used in system programming, embedded systems, and applications where performance is very important.

LAB EXERCISE: Research and provide three real-world applications where C programming is extensively used, such as in embedded systems, operating systems, or game development.

- Embedded Systems
  - C is used in devices like washing machines, microwaves, and IoT systems because it can directly control hardware and uses less memory.
- Operating Systems
  - Operating systems like Linux and UNIX are mostly written in C because it provides fast execution and better memory management.
- Game Development
  - C is used in game development for high performance and speed, especially in game engines and graphics processing.

- Setting Up Environment

THEORY EXERCISE: Describe the steps to install a C compiler (e.g., GCC) and set up an Integrated Development Environment (IDE) like DevC++, VS Code, or CodeBlocks.

Steps to Install C Compiler and IDE

- Download Compiler (GCC)

Install GCC compiler using MinGW (Windows) or directly on Linux/Mac.

- Set Environment Path

Add the compiler path (bin folder) to system environment variables.

- Install IDE

Download and install any IDE like Dev-C++, CodeBlocks, or VS Code.

- Configure IDE

Link the IDE with GCC compiler (select compiler path in settings).

- Test Installation

Write a simple program and compile it to check if everything works properly.

LAB EXERCISE: Install a C compiler on your system and configure the IDE. Write your first program to print "Hello, World!" and run it.

- Install Compiler and IDE

Install MinGW (GCC compiler) Install VS Code or CodeBlocks Configure compiler in IDE

- Write First Program `#include <stdio.h> int main() {`

```
    printf("Hello, World!");
    return 0;
}
```

- Compile and Run
  - Save file as `hello.c`
  - Compile using `gcc hello.c`
  - Run using `./a.out` or hello.exe
  - Basic Structure of a C Program

THEORY EXERCISE: Explain the basic structure of a C program, including headers, main function, comments, data types, and variables. Provide examples.

C program has the following main parts:

- Header Files

Used to include libraries.

Example: `#include <stdio.h>`

- Main Function

Execution of program starts from `main()` function. Example:

```
int main() {
    // code
}
```

- Comments

Used to explain code.

- Single line: `// comment`
- Multi-line: `/* comment */`

- Data Types

Define type of data.

Examples:

- `int` → integer
- `float` → decimal number
- `char` → character

- Variables

Used to store values.

Example:

```
int a = 10;
float b =
5.5; char c =
'A';
```

March 24, 2026

LAB EXERCISE: Write a C program that includes variables, constants, and comments. Declare and use different data types (int, char, float) and display their values.

```
#include <stdio.h>

// This program shows variables, constants, and data
types int main()
```

```
{
    // Variable declaration
    int age = 20;

    float height = 5.8;
    char grade = 'A';


    // Constant declaration
    const int year = 2026;


    // Display values
    printf("Age: %d\n",
    age);

    printf("Height: %.1f\n",
    height); printf("Grade: %c\n",
    grade); printf("Year: %d\n",
    year);


    return 0;

}
```

March 24, 2026

- Operators in C

THEORY EXERCISE: Write notes explaining each type of operator in C: arithmetic, relational, logical, assignment, increment/decrement, bitwise, and conditional operators..

- Arithmetic Operators

Used for mathematical calculations.
Examples: `+ , - , * , / , %`

- Relational Operators

Used to compare values.

Examples: `== , != , > , < , >= , <=`

Result is true (1) or false (0)

- Logical Operators

Used to combine conditions.

Examples: `&&` (AND), `||` (OR), `!` (NOT)

- Assignment Operators

Used to assign values to variables.
Examples: `=` , `+=` , `-=` , `*=` , `/=`

- Increment/Decrement Operators

Used to increase or decrease value by 1.
Examples: `++` , `--`

- Bitwise Operators

Work on binary values (bits).

Examples: `&` , `|` , `^` , `~` , `<<` , `>>`

- Conditional Operator (Ternary)

Short form of if-
else.

Example:

```
(condition) ? true_value : false_value;
```

March 24, 2026

LAB EXERCISE: Write a C program that accepts two integers from the user and performs arithmetic, relational, and logical operations on them. Display the results.

```
#include
<stdio.h> int
main() {

    int a, b;


    printf("Enter two integers: ");
    scanf("%d %d", &a, &b);


    // Arithmetic operations
    printf("Addition: %d\n", a + b);
    printf("Subtraction: %d\n", a - b);
    printf("Multiplication: %d\n", a *
    b); printf("Division: %d\n", a /
    b);
```

```
    // Relational operations
    printf("a > b: %d\n", a >
    b); printf("a == b: %d\n", a
    == b); printf("a != b:
    %d\n", a != b);


    // Logical operations

    printf("(a > 0 && b > 0): %d\n", (a > 0 && b >
    0)); printf("(a > 0 || b > 0): %d\n", (a > 0
    || b > 0)); printf("!(a > b): %d\n", !(a >
    b));


    return 0;

}
```

March 24, 2026

- Control Flow Statements in C

THEORY EXERCISE: Explain decision-making statements in C (if, else, nested if-else, switch). Provide examples of each.

Decision-Making Statements in C

- if Statement

Used to execute code when a condition is true.

```
if (a > b) {
    printf("A is greater");
}
```

- if-else Statement

Used when there are two choices.

```
if (a > b) {
```

```
    printf("A is greater");
} else {
    printf("B is greater");
}
```

- Nested if-else

if-else inside another if-else for multiple conditions.

```
if (a > b) {
    if (a > c) {
        printf("A is largest");
    }
}
```

- switch Statement

Used to select one option from many.

```
switch (num) {
    case   1:   printf("One");
    break;       case       2:
    printf("Two");       break;
    default:
    printf("Invalid");
}
```

March 24, 2026

LAB EXERCISE: Write a C program to check if a number is even or odd using an if-else statement.

Extend the program using a switch statement to display the month name based on the user's input (1 for January, 2 for February, etc.)

```
#include
<stdio.h> int
main() {
```

```
int num, month;


// Even or Odd
printf("Enter a number:
"); scanf("%d", &num);


    if (num % 2 == 0) {
    printf("Number is
    Even\n");

} else {

    printf("Number is Odd\n");

}


// Month using switch

printf("Enter month number (1-12):
"); scanf("%d", &month);


switch (month) {

    case 1: printf("January");
    break; case 2:
    printf("February"); break; case
    3: printf("March"); break; case
    4: printf("April"); break; case
    5: printf("May"); break;

    case 6: printf("June"); break;
    case 7: printf("July"); break;
    case 8: printf("August");
    break;

    case 9: printf("September");
    break; case 10: printf("October");
    break; case 11:
    printf("November"); break; case
    12: printf("December"); break;
    default: printf("Invalid month");

}


return 0;
```

```
}
```

March 24, 2026

- Looping in C

THEORY EXERCISE: Compare and contrast while loops, for loops, and do-while loops. Explain the scenarios in which each loop is most appropriate.

Comparison of Loops

- **while Loop**
  - Checks condition before execution
  - Used when number of iterations is not known

```
while (condition) {
    // code
}
```

- for Loop
  - Used when number of iterations is known
  - Initialization, condition, update in one line

```
for (i = 1; i <= 10; i++) {
    // code
}
```

- do-while Loop
  - Executes at least once (condition checked after)
  - Useful when loop must run at least one time

```
do {
```

```
        // code
    } while (condition);
```

March 24, 2026

LAB EXERCISE: Write a C program to print numbers from 1 to 10 using all three types of loops (while, for, do-while).

```
#include <stdio.h>


int main() {
    int i;


    // while
    loop i = 1;
        while (i <= 10)
        { printf("%d ",
        i); i++;
    }


    printf("\n");


    // for loop
        for (i = 1; i <= 10; i++) {
        printf("%d ", i);
    }


    printf("\n");


    // do-while
    loop i = 1;
    do {
        printf("%d ",
        i); i++;
```

```
    } while (i <= 10);

    return 0;
}
```

March 24, 2026

- Loop Control Statements

THEORY EXERCISE: Explain the use of break, continue, and goto statements in C. Provide examples of each.

- break Statement
  - Used to exit the loop immediately

```
    if (i ==
    5) {
    break;
}
```

- continue Statement
  - Skips current iteration and moves to next

```
if (i == 3) {
    continue;
}
```

- goto Statement
  - Transfers control to a labeled statement

```
goto label;
label:
printf("Hello")
;
```

March 24, 2026

LAB EXERCISE: Write a C program that uses the break statement to stop printing numbers when it reaches 5. Modify the program to skip printing the number 3 using the continue statement.

```
#include
<stdio.h> int
main() {
    int i;

    // Using break
        for (i = 1; i <= 10;
        i++) { if (i == 5) {
            break;
        }
        printf("%d ", i);
    }

    printf("\n");

    // Using continue
        for (i = 1; i <= 10;
        i++) { if (i == 3) {
            continue;
        }
        printf("%d ", i);
    }

    return 0;
}
```

March 24, 2026

Q-8. Functions in C

THEORY EXERCISE: What are functions in C? Explain function declaration, definition, and how to call a function. Provide examples.

Functions are blocks of code used to perform a specific task and improve reusability.

- Declaration → tells function name and parameters
- Definition → actual code of function
- Call → executes the function

```
int add(int a, int b); // declaration

    int add(int a, int b) { //
    definition return a + b;

}


int main() {

    int sum = add(2, 3); // call

}
```

LAB EXERCISE: Write a C program that calculates the factorial of a number using a function. Include function declaration, definition, and call.

```
#include <stdio.h>

int fact(int n); //
declaration int fact(int n) {
// definition

    int i, f = 1;

        for(i = 1; i <= n;
        i++) f *= i;

    return f;

}


 int main()
          {

   int num;
```

```
    printf("Enter number: ");
    scanf("%d", &num);


    printf("Factorial = %d", fact(num)); // call
    return 0;
}
```

March 24, 2026

Q-9. Arrays in C

THEORY EXERCISE: Explain the concept of arrays in C. Differentiate between one-dimensional and multi-dimensional arrays with examples.

Array = collection of same type elements.

1D Array

```
int a[5] = {1,2,3,4,5};
```

2D Array

```
int a[3][3];
```

Difference:

- 1D → single row
- 2D → rows + columns (matrix)

March 24, 2026

LAB EXERCISE: Write a C program that stores 5 integers in a one-dimensional array and prints them. Extend this to handle a two-dimensional array (3x3 matrix) and calculate the sum of all elements.

```
#include
<stdio.h> int
main() {

    int i, j, sum = 0;

    int a[5] = {1,2,3,4,5};


    // 1D array print
    for(i = 0; i < 5;
    i++)

    printf("%d ", a[i]);
    printf("\n");

    // 2D array

    int b[3][3] = {

        {1,2,3},

        {4,5,6},

        {7,8,9}

    };


        for(i = 0; i < 3; i++)
        { for(j = 0; j < 3;
        j++) {

            sum += b[i][j];

        }

    }


    printf("Sum = %d",
    sum); return 0;

}
```

March 24, 2026

Q-10. Pointers in C

THEORY EXERCISE: Explain what pointers are in C and how they are declared and initialized. Why are pointers important in C?

Pointers store the address of a variable.

```
int a = 10;
int *p = &a;
```

Importance:

Direct memory access

Used in arrays, functions, dynamic memory

LAB EXERCISE: Write a C program to demonstrate pointer usage. Use a pointer to modify the value of a variable and print the result.

```
#include <stdio.h>


int main() {

    int a = 10;
    int *p = &a;


    *p = 20; // modify using pointer


    printf("Value of a = %d", a);
    return 0;

}
```

March 24, 2026

Q-11. Strings in C

THEORY EXERCISE: Explain string handling functions like strlen(), strcpy(), strcat(), strcmp(), and strchr(). Provide examples of when these functions are useful.

`strlen()` → length
`strcpy()` → copy
`strcat()` → join strings
`strcmp()` → compare
`strchr()` → find
character.

LAB EXERCISE: Write a C program that takes two strings from the user and concatenates them using strcat(). Display the concatenated string and its length using strlen().

```
#include <stdio.h>
#include
<string.h>


int main() {
    char s1[50], s2[50];


    printf("Enter first string: ");
    scanf("%s", s1);


    printf("Enter second string:
    "); scanf("%s", s2);


    strcat(s1, s2);


    printf("Concatenated string: %s\n", s1);
    printf("Length: %lu", strlen(s1));


    return 0;
}
```

March 24, 2026

Q-12. Structures in C

THEORY EXERCISE: Explain the concept of structures in C. Describe how to declare, initialize, and access structure members.

Structure = collection of different data types.

```
    struct
    Student {
    char
    name[20];
    int roll;
    float marks;
};
```

LAB EXERCISE: Write a C program that defines a structure to store a student's details (name, roll number, and marks). Use an array of structures to store details of 3 students and print them.

```
#include <stdio.h>


    struct
    Student {
    char
    name[20];
    int roll;
    float marks;
};


int main() {
    struct Student s[3];


    for(int i=0;i<3;i++) {
        printf("Enter name, roll, marks: ");
        scanf("%s %d %f", s[i].name, &s[i].roll, &s[i].marks);
    }


    for(int i=0;i<3;i++) {
        printf("%s %d %.2f\n", s[i].name, s[i].roll, s[i].marks);
    }


    return 0;
}
```

March 24, 2026

Q-13. File Handling in C

THEORY EXERCISE: Explain the importance of file handling in C. Discuss how to perform file operations like opening, closing, reading, and writing files. .

`fopen()` → open file
`fclose()` → close file
`fprintf()` → write
`fscanf()` → read

LAB EXERCISE: Write a C program to create a file, write a string into it, close the file, then open the file again to read and display its contents.

```
#include <stdio.h>


int main() {

    FILE *fp;


    // write

    fp = fopen("test.txt", "w");
    fprintf(fp, "Hello World");
    fclose(fp);


    // read

    char str[50];

    fp = fopen("test.txt", "r");

    fscanf(fp, "%s", str);

    printf("Data: %s", str);
    fclose(fp);


    return 0;

}
```